



Technological Institute of the Philippines - Manila Campus

Systems Integration and Architecture 1

**OpenDoor: An Integrated, Accommodation-Aware Job Matching and Employer
Accountability Platform for Persons with Disabilities**

Submitted by:

Jianina Marie M. Beronilla

Angellana P. Crisologo

Alfred G. Su

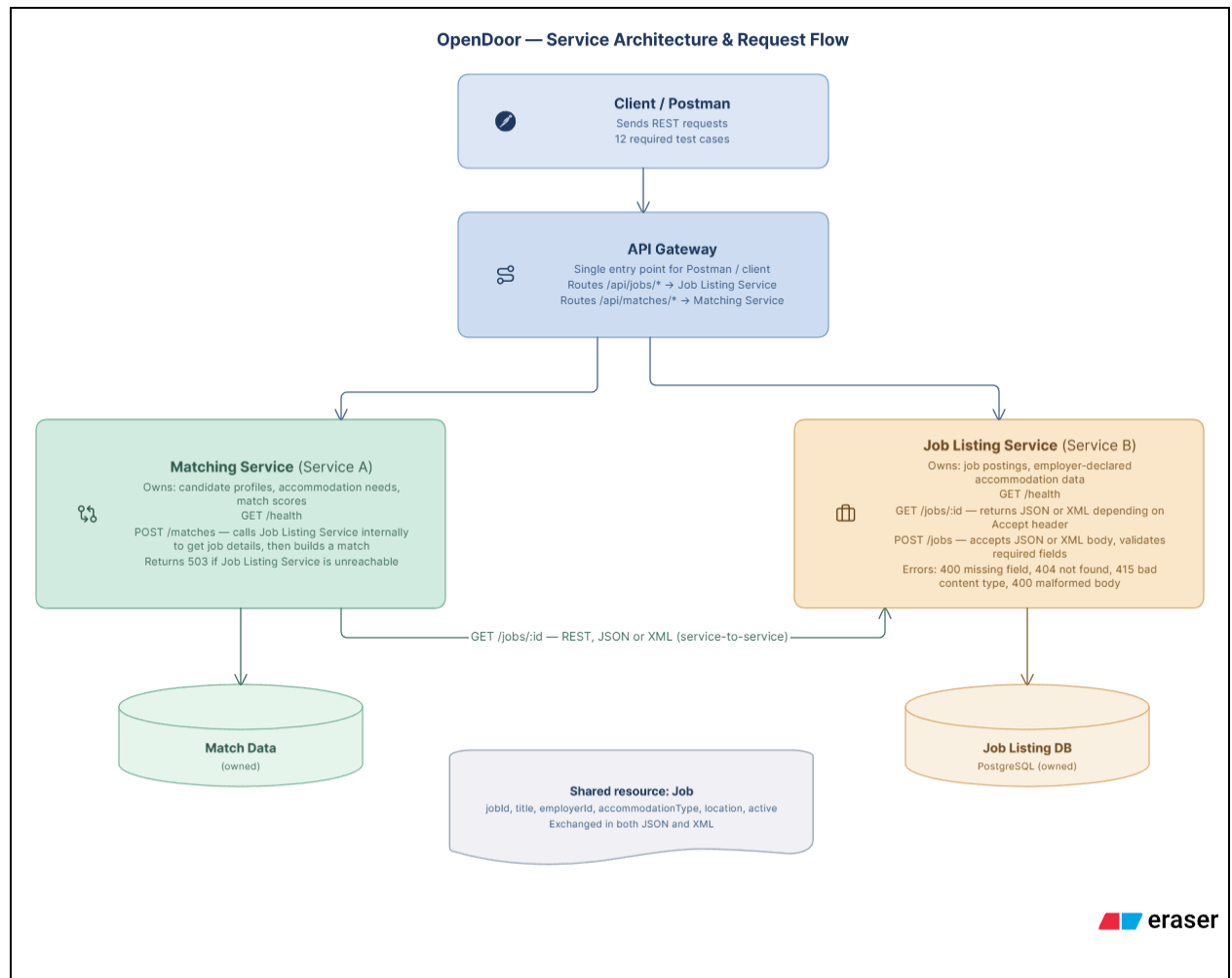
Francis Aaron R. Ruzgal

OneKhyle Siochi

IT31S1 - Group 3

I. DEFINE YOUR MICROSERVICE ARCHITECTURE

ITEM	MICROSERVICE A	MICROSERVICE B
Service Name	Matching Service	Job Listing Service
Business Responsibility	Takes a candidate's accommodation needs, calls Job Listing Service to get a job's details, and builds a match between the two	Owns and serves job postings — creating, retrieving, and validating job data, including employer-declared accommodation details
Main Resource/Data	Candidate profiles, accommodation needs, match scores (Match Data)	Job postings, employer-declared accommodation data.
Example Operations	GET /health, POST /matches (internally calls GET /jobs/:id on Job Listing Service before creating the match record)	GET /health, GET /jobs/:id (returns JSON or XML depending on Accept header), POST /jobs (accepts JSON or XML body, validates required fields)
Why Separate?	Because it doesn't own job data — it consumes it. If Matching Service and Job Listing Service were the same service, every job lookup would just be a local function call, and the lab's actual requirement couldn't exist. Separating them is what forces — and proves — the service-to-service integration pattern the lab is testing.	Job Listing Service changes when job-posting rules change (new required fields, new validation). Matching Service changes when matching logic changes (scoring, ranking). Bundling them means a change to one risks breaking or redeploying the other for no reason — keeping them apart means each can be built, tested, deployed, and scaled independently.



II. DESIGN THE REST API

Job Listing Service (services/job-listing, port 5001)

Method	Endpoint	Purpose	Input	Response
GET	/health	Health check	—	{ "status": "UP", "service": "job-listing" }

GET	/jobs/:id	Retrieve a job by ID; supports content negotiation	Accept: application/json or application/xml	Job resource in requested format, or 404
POST	/jobs	Create a new job	JSON or XML body with title, employerId required	201 with created job, or 400/415 on error

Matching Service - (services/matching, port 5001)

SERVICE	METHOD	ENDPOINT	PURPOSE	RESPONSE
Matching	GET	/health	Checks whether the service is running	200 service status JSON (UP)
Matching	POST	/matches	Main integration endpoint. Creates a match record for a candidate by calling Job Listing Service (GET /jobs/:id) to fetch job details and confirm active status	JSON body: candidateId, jobId 201 created match record, 400 missing field, 404 if job not found, 409 if inactive, 503 if Job Listing Service is unreachable

API Gateway - <http://localhost:5000>

METHOD	ENDPOINT	ROUTE & TO
GET	/health	Gateway's own status check (200 JSON: UP)
GET / POST	/api/jobs	Job Listing Service (http://localhost:5001/jobs)
GET	/api/jobs/:id	Job Listing Service (http://localhost:5001/jobs/:id)
POST	/api/matches	Matching Service (http://localhost:5002/matches)

III. IMPLEMENT THE MICROSERVICES

Both services are implemented as independently runnable Node.js/Express applications:

- **Job Listing Service** – runs standalone on **<http://localhost:5001>**
- **Matching Service** – runs standalone on **<http://localhost:5002>**
- **API Gateway** – runs standalone on **<http://localhost:5000>** and **centrally routes to both**

Each service maintains its own isolated in-memory datastore—the **Job Listing Service** exclusively manages job postings and employer accommodation specifications, while the **Matching Service** independently stores candidate profiles and generated match records. This adheres directly to the database-per-service pattern, preventing direct database access across service boundaries and requiring cross-service retrieval to occur via HTTP API calls.

Additionally, both microservices expose a standalone GET /health endpoint (<http://localhost:5001/health> and <http://localhost:5002/health>) to verify independent runtime availability. This was validated during testing by executing individual health check requests in Postman to ensure both Node.js services were responsive prior to executing routing and integrated match requests through the API Gateway.

IV. DESIGN THE DATA CONTRACT

Data Contract Description for Your Report

- **Resource Name:** Job
- **Primary Identifier:** jobId (Integer)
- **Domain Ownership:** Exclusively stored and managed by the **Job Listing Service** (<http://localhost:5001>).
- **Consumer Service:** Retrieved by the **Matching Service** (<http://localhost:5002>) over HTTP during candidate accommodation evaluation.
- **Fields & Data Types:**
 - jobId (Integer, required) — Unique system identifier for the job post.
 - title (String, required) — Role title / job designation.
 - employerId (Integer, required) — Identifier of the employer who created the posting.
 - accommodationType (String, optional/default: "Not specified") — Workplace accommodation provided (e.g., "Flexible schedule", "Wheelchair accessible facility").
 - location (String, optional/default: "Not specified") — Worksite location or city.
 - active (Boolean) — Availability status indicator (true/false) evaluated by the matching transaction before establishing candidate pairings.

resource.json	resource.xml
opendoor / docs / examples / resource.json ryujiinnn08 chore: align repo structure with lab requirements, Code Blame 8 lines (8 loc) · 169 Bytes 1 { 2 "jobId": 1001, 3 "title": "Data Entry Clerk", 4 "employerId": 501, 5 "accommodationType": "Flexible schedule", 6 "location": "Manila", 7 "active": true 8 }	ryujiinnn08 chore: align repo structure with lab requirements, add REAL Code Blame 9 lines (9 loc) · 251 Bytes 1 <?xml version="1.0" encoding="UTF-8"?> 2 <job> 3 <jobId>1001</jobId> 4 <title>Data Entry Clerk</title> 5 <employerId>501</employerId> 6 <accommodationType>Flexible schedule</accommodationType> 7 <location>Manila</location> 8 <active>true</active> 9 </job>

V. IMPLEMENT JSON DATA EXCHANGE

Ingress (Accepts application/json):

POST /jobs in services/job-listing/server.js uses `express.json()` to parse incoming JSON payloads (deserialization $\rightarrow$ validates title and employerId $\rightarrow$ pushes newJob).

Egress (Serializes to JSON):

GET /jobs/:id in services/job-listing/server.js uses `res.json(job)`.

VI. IMPLEMENT XML DATA EXCHANGE

Ingress (Accepts application/xml):

POST /jobs in services/job-listing/server.js uses `express.text({ type: 'application/xml' })` along with `xml2js.parseStringPromise()` to deserialize XML into application data prior to validation.

Egress (Serializes to XML):

Uses `xml2js.Builder({ rootName: 'job', headless: false })` to serialize the newJob or retrieved job object back into well-formed XML if requested or submitted as XML.

VII. IMPLEMENT CONTENT NEGOTIATION

Dual Representation on GET:

GET /jobs/:id inspects `req.headers.accept`:

- If `req.headers.accept.includes('application/xml')`, sets `Content-Type: application/xml` and returns `xmlBuilder.buildObject(job)`.
- Otherwise defaults to `Content-Type: application/json` via `res.json(job)`.

VIII. IMPLEMENT SERVICE-TO-SERVICE INTEGRATION

Service A (Consumer): Matching Service on :5002 receiving POST /matches.

Service B (Provider): Job Listing Service on :5001.

Flow:

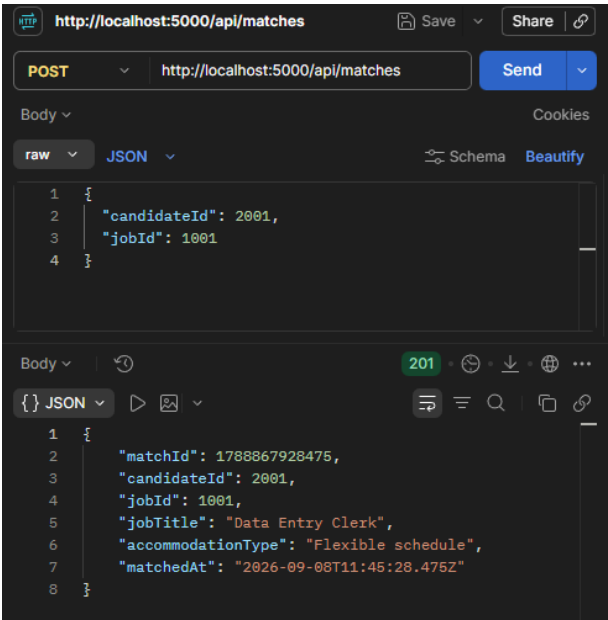
1. Client sends POST /api/matches with { "candidateId": 2001, "jobId": 1001 }.
2. Matching Service initiates HTTP GET http://localhost:5001/jobs/1001 with Accept: application/json via axios.
3. Job Listing Service processes the request, serializes job 1001 to JSON, and responds 200 OK.
4. Matching Service deserializes the JSON response, validates job.active === true, and combines the candidate data with job.title and job.accommodationType into a new match record (201 Created).

IX. API GATEWAY / CENTRALIZED ROUTING

```
PS C:\Systems\SIA-2026\opendoor\gateway> node server.js  
API Gateway running on port 5000  
|
```

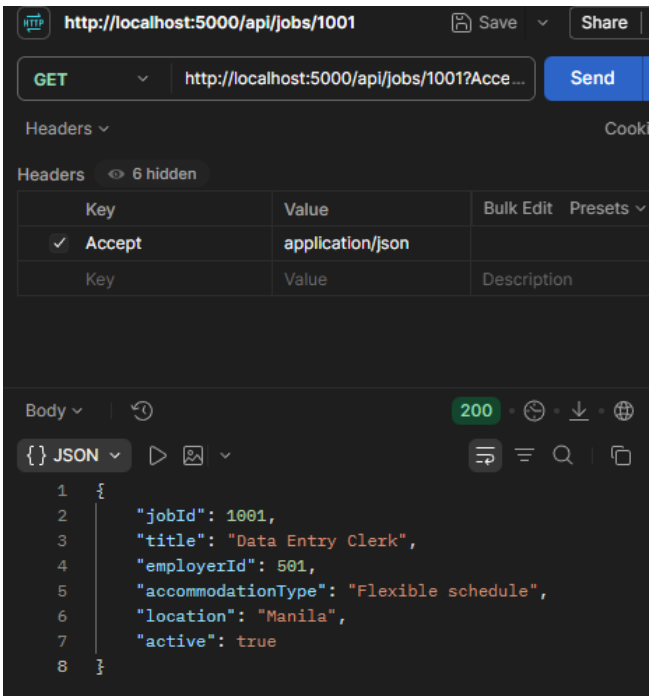
201 Created (POST /api/matches)

Indicates successful creation of a match record between candidate 2001 and job 1001, returning the generated matchId alongside matched details and a creation timestamp.



200 OK (GET /api/jobs/1001)

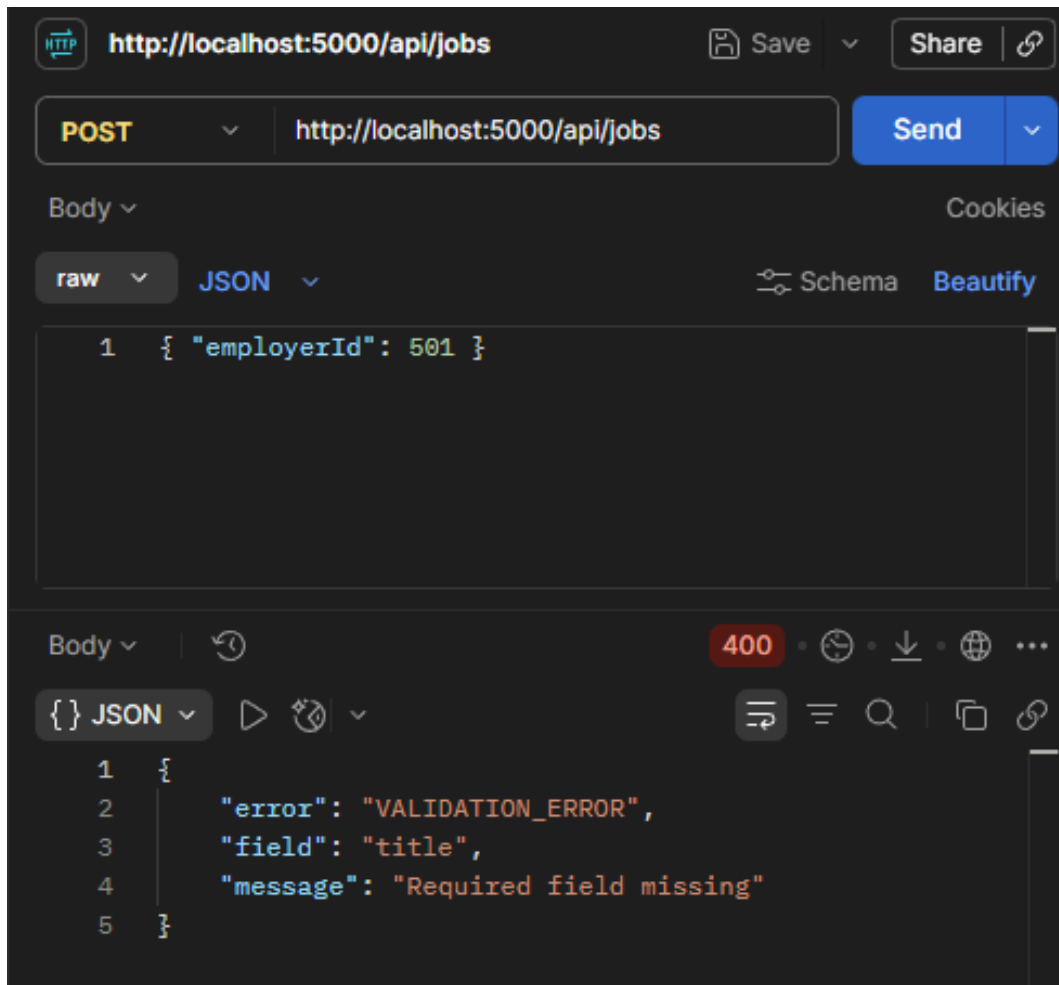
Indicates a successful retrieval of an existing job record, returning the complete job resource payload in JSON format according to the requested Accept: application/json header.



X. VALIDATION AND ERROR HANDLING

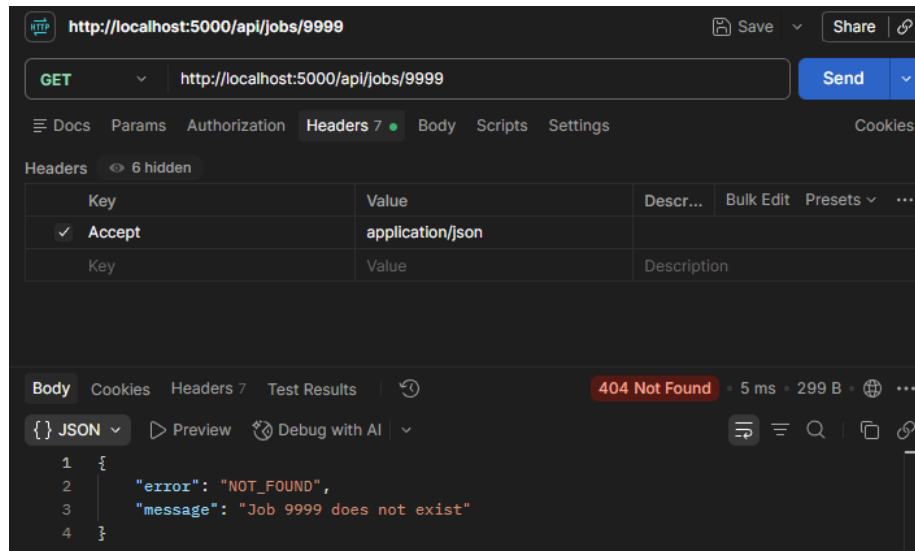
400 Bad Request (VALIDATION_ERROR)

Occurs when a required attribute is missing from the payload. In this case, POST /api/jobs failed validation because the mandatory title field was not provided.



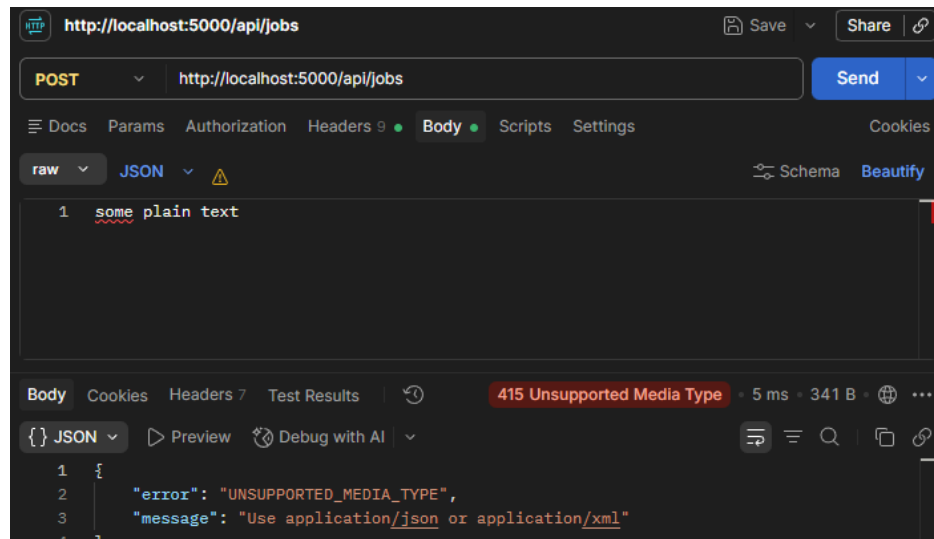
404 Not Found (NOT_FOUND)

Occurs when querying an identifier that does not exist in the database. GET /api/jobs/9999 returns this error indicating no record matches the provided Job ID.



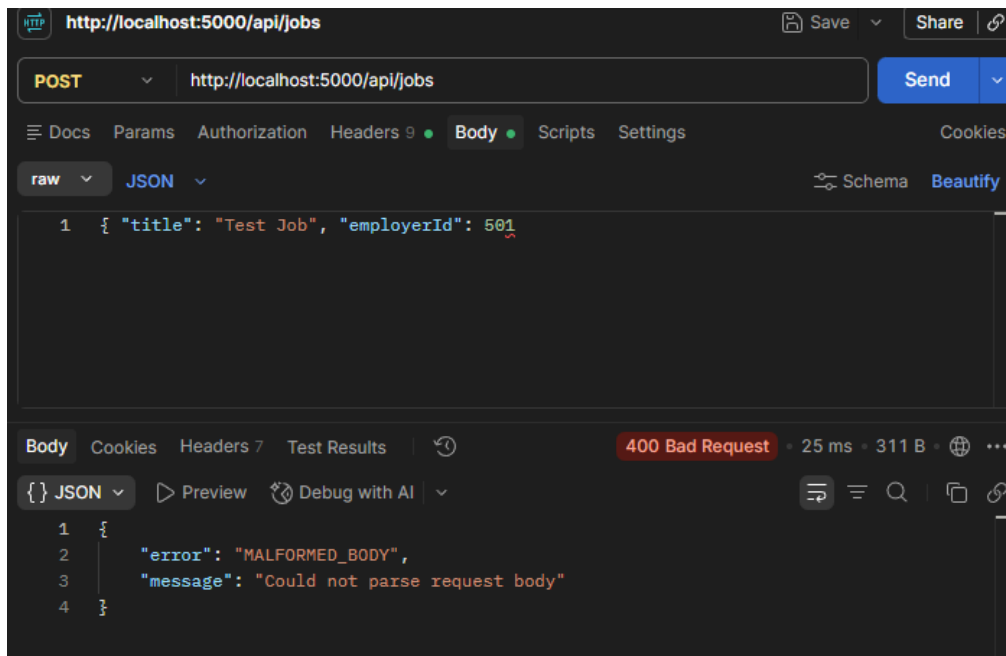
415 Unsupported Media Type (UNSUPPORTED_MEDIA_TYPE)

Occurs when the request payload format is not supported by the API. The server requires application/json or application/xml instead of plain text.



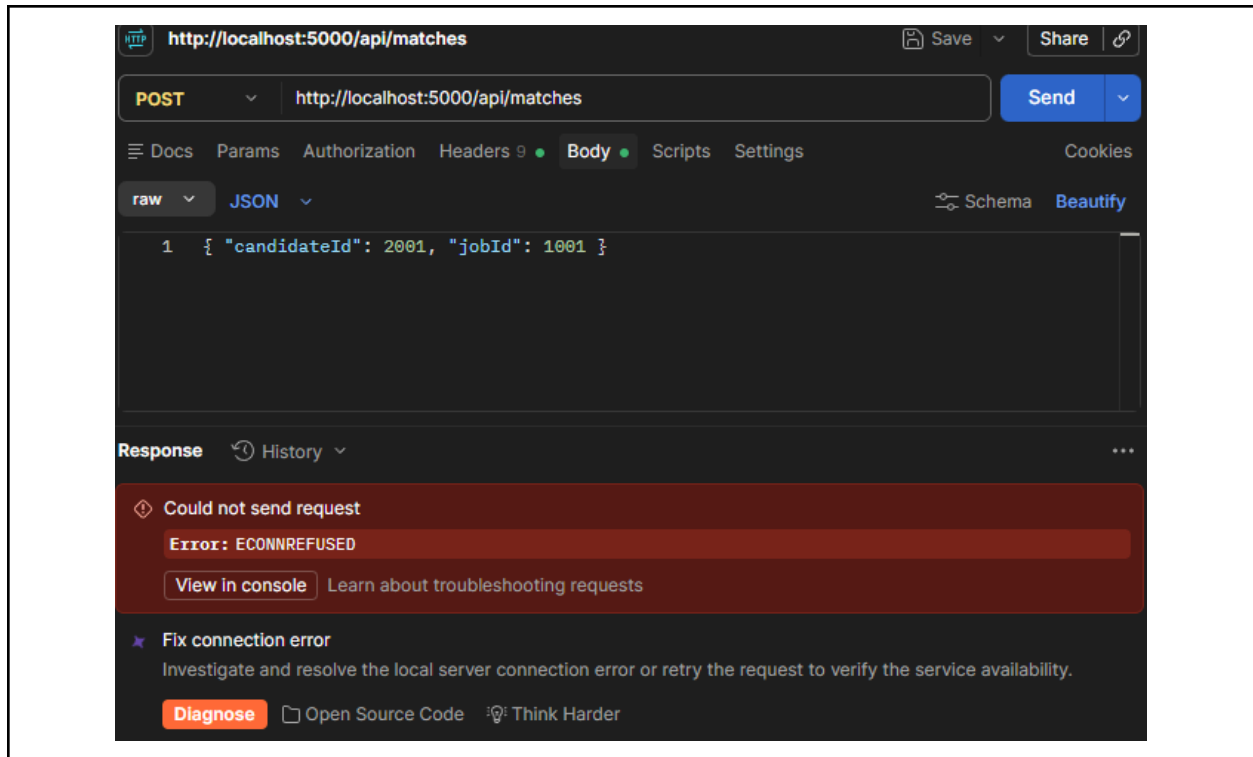
400 Bad Request (MALFORMED_BODY)

Occurs when the request body cannot be parsed due to invalid syntax (such as a missing closing brace or broken JSON structure).



Client Connection Error (ECONNREFUSED)

Occurs when Postman cannot establish a TCP connection to the destination host (localhost:5000), typically because the API gateway or backend service is offline or not running on the specified port.



XI. POSTMAN INTEGRATION TESTING

The Postman collection (postman/collection.json) demonstrates all required scenarios:

1. Job Listing Health
2. Matching Health
3. Get Job - JSON
4. Get Job - XML
5. Create Job - JSON
6. Missing Required Field
7. Malformed JSON
8. Job Not Found
9. Unsupported Media Type
10. Create Match - Integrated Transaction

The requests cover health checks, multi-format content negotiation (JSON and XML), resource creation, schema validation, structured error handling, and service-to-service integrated workflows. All scenarios were executed and verified successful during development.

XII. GITHUB REPOSITORY AND DOCUMENTATION

Github Repository: <https://github.com/ryujiinnn08/opendoor>

